

DASS MIDSEM RUBRIC

q1-6 - narain q7 - divijh q8 - george, santosh q9 - farzana, rudra q10 - nikhilesh, vijay q11 - neel, parth q12 - yashaswinee, aniket, karthik q13 - agrim, aman, agyeya

Question 1 [1 marks]

B

Question 2 [1 marks]

B

Question 3 [1 marks]

D

Question 4 [1 marks]

any assert method such assertTrue(condition) or assertEquals(output, actual) should work in any particular language

Question 5 [1 marks]

B

Question 6 [2 marks]

Yes the senior engineer is correct. (if not written, straight 0)

Implementation uses long conditional chain (0.5 marks), everytime a new shipping type is added, the method is modified hence violating OCP (0.5 marks).

Any one of the following solution is accepted:

If shipping methods is treated as a variable - hash map or any lookup mechanism works (1 marks) If shipping methods are treated as a class - strategy pattern works (1 marks)

Question 7 [2 marks]

Drivers are cheaper to write because they **only invoke the module being tested and provide input data**. [1 mark]

In contrast, **stubs must simulate the behavior and outputs of missing lower-level modules**, which requires more logic and implementation. [1 mark]

Therefore, stubs are more complex and time-consuming to develop than drivers.

Wrong Answers[0 marks]:

- Because lesser in number - incorrect as that depends on structure and you could have more drivers than stubs (ex. A->B; B->C,D,E,F,G;(C,D,E,F,G are independent and work differently in B - think C,D,E,F as different types of supported payments) each will need separate driver to test it). The **main differentiator is complexity of writing** (which is lesser for drivers as explained above) regardless of quantity. Could be

easier to write more of something that is less complex than write less of something more complex. Quantity is not a generalisable answer.

- Lower modules get repeatedly tested - that is true but doesn't explain why it's considered more "*cheaper to write*".
- Easier to find bugs - Question asks about *cheaper to write*, not testing effectiveness.
- Computation/Memory/Performance/System getting completed - irrelevant.

Question 8 [4 marks]

- Each example with proper justification is two marks.
- Matching with some requirement in FinTech Company that would like not be easily detectable by any of the previous phases but will be caught by an experienced developer should be mentioned.
- Some ideas could be like higher latency, change of policies in the company regarding what all to invest, mistakes being very costly in FinTech and very hard to recover since no option to recall whatever was done, maybe the current news changed and thus the current system designed is not valid anymore, etc

Question 9 [4 marks]

Integration testing (checks how modules work together) and **System testing** (validates the full system under realistic conditions) should have caught this defect.

Both failed due to a lack of concurrency in the test suite. All related tests were sequential. No test ever sent two updates to Module B at the same time while Module C was reading from it, so the race condition was never triggered.

The specific test scenario that would have exposed this bug involves sending concurrent update requests to Module B while Module C reads the same record, verifying that Module C always receives the latest committed value.

Question 10 [6 marks] - Vijay and Nikhilesh

Question: Develop an automated ordering system for the The Most Amazing Coffee Shop, where customers can place orders at self-service kiosks (SSKs) located inside the shop. Each order will include the customer's selection of drinks (e.g., espresso, latte, cappuccino), size (small, medium, large), any added customizations (e.g., milk type, flavor shots), and payment information. Once an order is placed and payment is confirmed, the system will generate a receipt, and the customer will be notified when their order is ready for pickup. The coffee shop staff will also be able to update the menu and availability of items in real-time. Note: You may make simple assumptions about this problem based on your own knowledge of coffee shop ordering systems. a) (3 points) Draw the usecase model for the automated ordering system clearly showing all the usecases, primary and supporting actors. b) (3 points) Write a detailed textual use case (step-by-step) for a customer successfully placing an order using the SS

Answer

a) Rubric for use case diagram (3 marks)

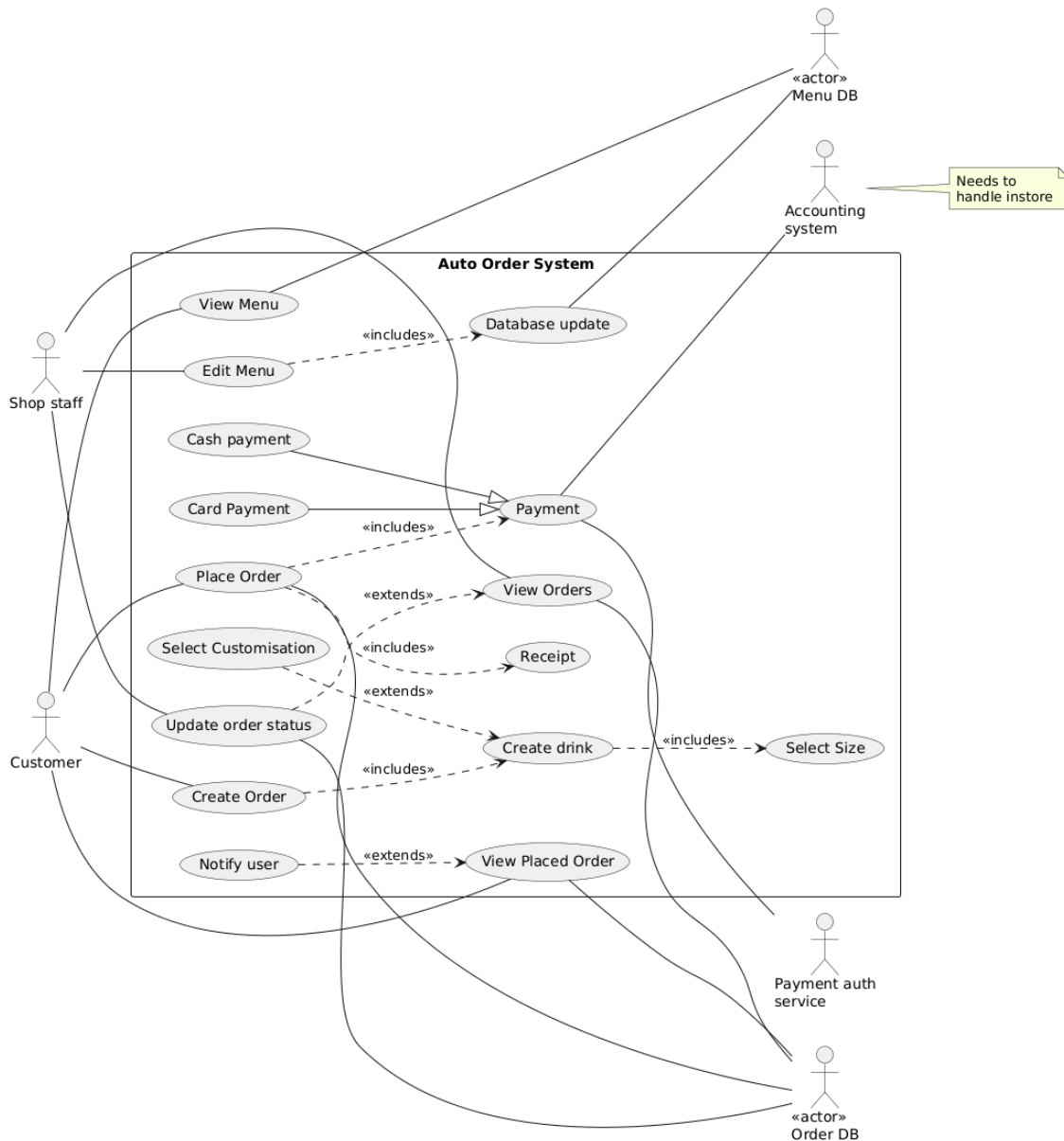
Ans

- 0.5 mark for correct formatting (Name, arrow direction, actor format, box etc.)
 - 0.25 for 1-2 consistent mistakes
 - 0 for >2 mistakes
- 0.5 mark for the actors
- 0.5 mark for the external actors
- 1 mark for all core customer functionality (at least 2 includes (for the size and the customisation))
- 1 mark for all shop staff functionality

- 0.5 for innovative parts like specialization or some thing

b) Rubric for use case description

- 1 mark for correct formatting
 - Presence of all the fields (Except special properties)
- 1 mark for logical main flow
- 0.5 mark for ≥ 1 sub flow, ≥ 1 alt flow
- 0.5 for post condition



One possible solution (Please check once): <https://1drv.ms/o/c/bf6be0865a77ea24/IgDOu4qz-7FWQp5Inrw31DUQAeZ98I14MfGVtyJT2U6zZyl>

Question 11 [6 marks]

Part A: Calculate CPI and SPI (2 Points)

Cost Performance Index (CPI) (1 point total)

- **0.5 points:** Correct formula stated or implied: $CPI = EV / AC$
- **0.5 points:** Correct calculation and final answer: $CPI = 48,000 / 54,000 \approx 0.89$

Schedule Performance Index (SPI) (1 point total)

- **0.5 points:** Correct formula stated or implied: $SPI = EV / PV$
- **0.5 points:** Correct calculation and final answer: $SPI = 48,000 / 60,000 = 0.80$

Part B: Identify Data Reveals & Evaluate Statements (2 Points)

Evaluating the Sponsor's Statement (1 point total)

- **0.5 points:** Correct conclusion: The sponsor is **incorrect**.
- **0.5 points:** Proper justification: The sponsor is confusing Actual Cost (AC) with the overall budget (BAC) without looking at what value was actually delivered (Earned Value). Because the CPI is less than 1 (0.89), the project is currently **over budget** for the work performed. They spent \$54,000 to only get \$48,000 worth of work.

Evaluating the Development Lead's Statement (1 point total)

- **0.5 points:** Correct conclusion: The development lead is **incorrect**.
- **0.5 points:** Proper justification: Finishing one specific module early does not reflect the status of the entire project. The SPI is less than 1 (0.80), which means overall, the project is completing tasks at only 80% of the planned rate and is currently **behind schedule**.

Part C: Actions for Next Sprint Planning (2 Points)

Actions like - Root Cause Analysis, Corrective Communication, Re-estimation, Scope Adjustment / Prioritization, Process Improvement etc. any two distinct, practical project management actions based on the negative EVM indicators. 1 point per valid action (up to 2 points max).

Question 12 [8 marks]

Marking Scheme – Question 12 (8 points)

(a) (2 points)

- **[1 mark]:** Curve A has a horizontal asymptote at 100% bug detection, meaning the curve gets closer and closer to 100% but never actually reaches it.
- **[1 mark]:** 100% bug detection would require an infinite number of test cases (or exhaustive testing of all possible inputs/states), which is impossible in practice due to the input space, time, and resource constraints.

(b) (2 points)

- **[1 mark]:** Arguing against the conclusion, statement coverage and bug detection are not equivalent metrics. 95% statement coverage does not mean 95% of bugs have been found.
 - **[1 mark]:** Statement can be executed without triggering a fault (e.g., wrong conditions, boundary values, or data combinations not tested). **Coverage measures code exercised, not correctness of behaviour.** Many bugs only manifest under specific data conditions or interactions not captured by statement coverage alone.
-

(c) (4 points)

Curve B – Random / Ad-hoc test suite (2 marks)

- **1 mark** The test suite hits many different code paths early, producing a steep mid-range rise as easily reachable bugs are quickly encountered through sheer volume of varied inputs. While deeper and more complex bugs are found occasionally in later stages of testing.
- **1 mark** Without risk-based or structured targeting, redundant and overlapping test cases accumulate rapidly, causing the curve to plateau well below Curve A, the remaining undetected bugs reside in corner cases and complex interactions that random selection is statistically unlikely to reach within a finite suite

Curve C – Coverage-driven (late-surge) suite (2 marks)

- **1 mark** Coverage-driven suites are constructed to satisfy structural criteria (e.g., branch or path coverage) rather than fault likelihood. Early test cases satisfy easy coverage targets that coincidentally exercise low-fault-density code, keeping bug yield per test case consistently low in the early stages.
- **1 mark** Significant bug detection is deferred until broad structural coverage has already been achieved, reflecting the strategy's indifference to fault probability.

Question 13 [8 marks]

Standard Weights Table

	Function Points		
Program Characteristic	Low Complexity	Medium Complexity	High Complexity
Number of Inputs	x 3	x 4	x 6
Number of Outputs	x 4	x 5	x 7
Inquiries	x 3	x 4	x 6
Logical internal files	x 7	x 10	x 15
External interface files	x 5	x 7	x 10

Complexity weights other than the ones above are allowed if they have been explicitly declared.

Calculating Unadjusted Function Points (UFP) - 3 marks

- Inputs = $(2 \times 3) + (3 \times 4) + (1 \times 6) = 24$
- Outputs = $(1 \times 4) + (2 \times 5) + (2 \times 7) = 28$
- Inquiries = $(4 \times 3) + (1 \times 4) + (0 \times 6) = 16$
- Internal Logical Files = $(1 \times 7) + (2 \times 10) + (1 \times 15) = 42$
- External Interface Files = $(3 \times 5) + (1 \times 7) + (0 \times 10) = 22$

Total UFP = 132

Calculating Adjusted Function Points (AFP) - 1 mark

Value adjustment factor = 1.1
\$\$\$ AFP = UFP * VAF
\$\$\$ AFP = $132 \times 1.1 = 145.2$; (or; 145) \$\$\$

Total Effort Required - 1 mark

\$\$\$ Effort = Function;Points * Hours;per;Point
\$\$\$ Effort = $145.2 \times 18 = 2613.6$; hours \$\$\$

Base Duration - 1 mark

Weekly Capacity = $4 * 30$ hours/week = 120 hours/week Base Duration = $2613.6 / 120 = 21.78$ weeks

Total Project Duration - 2 marks

Buffer = 15% Project Duration = $21.78 * 1.15 = 25.047$

Rounding off, Total Project Duration = 25 weeks

NOTE: Only writing the correct answer without showing the calculations will fetch 0 marks.